SCALABILITY AUDIT OF SHOPSPHERE

CSA06 – Design and Analysis of Algorithms | CO1 Assessment Tool 1

Student Name: Preethika M Register Number: 192421084

Assumption: One basic operation is counted as one operation, and the server executes 10^8 basic operations per second. Running-time estimates based on asymptotic expressions are approximate because the recurrence base-case costs and implementation constants are not specified.

(a) Understand the Problem

The input size n is the number of products, indexed records, or data items processed by each module. The dominant basic operation is the operation whose count grows fastest and therefore determines the asymptotic running time.

- M1 – Duplicate-Listing Detector: n products. The dominant operation is comparing the title and seller of a pair of products inside the nested loops. It executes once for every unordered pair, so the count grows quadratically: $\Theta(n^2)$.
- M2 – API Rate Limiter: n request records. The dominant operation is Audit(Request[j]). The inner-loop sizes double: 1, 2, 4, 8, …, while the outer loop runs logarithmically many times. The total is a geometric sum, $\Theta(n)$, not $\Theta(n \log n)$.
- M3 – Product Search S1: n products in an unsorted catalogue. The dominant operation is comparing the search key with each product. Its count ranges from 1 to n, giving $\Theta(1)$ best case and $\Theta(n)$ average/worst case.
- M3 – Product Search S2: n entries in a sorted index. The dominant operation is comparing the key with the middle entry. Each comparison discards half the remaining entries, giving $\Theta(\log n)$ time.
- M4 – Recommendation Engine: n customer–product records. The dominant work is merging partial results in linear time at each recursive level. The current recurrence has four subproblems of size n/2, while the redesign has two; their results are $\Theta(n^2)$ and $\Theta(n \log n)$, respectively.
- M5 – Sales Analytics Engine: n sales records. The dominant work per recursive level is n log n. Across the recursion tree, this produces $\Theta(n \log^2 n)$. The separate report generator has $f(n) = 3n^2 + 10n \log_2 n + 50$, dominated by its quadratic term, so it is $\Theta(n^2)$.

(b) Analyse the Iterative Modules

M1 – Exact number of comparisons

For each i = 0, 1, …, n − 2, the inner loop compares Product[i] with all later products. The number of comparisons for a fixed i is n − i − 1.

```
C(n) = (n − 1) + (n − 2) + … + 2 + 1 + 0
     = 1 + 2 + … + (n − 1)
     = n(n − 1) / 2
     = (n² − n) / 2
```

Therefore, the exact comparison count is $n(n-1)/2$ and the time complexity is $\Theta(n^2)$. The routine uses only loop variables and a fixed amount of extra working storage, so its auxiliary space complexity is $\Theta(1)$, excluding the catalogue and any storage used by MarkDuplicate.

At n = 100,000: C(n) = 100,000 × 99,999 / 2 = 4,999,950,000 comparisons.

M2 – Exact number of Audit operations

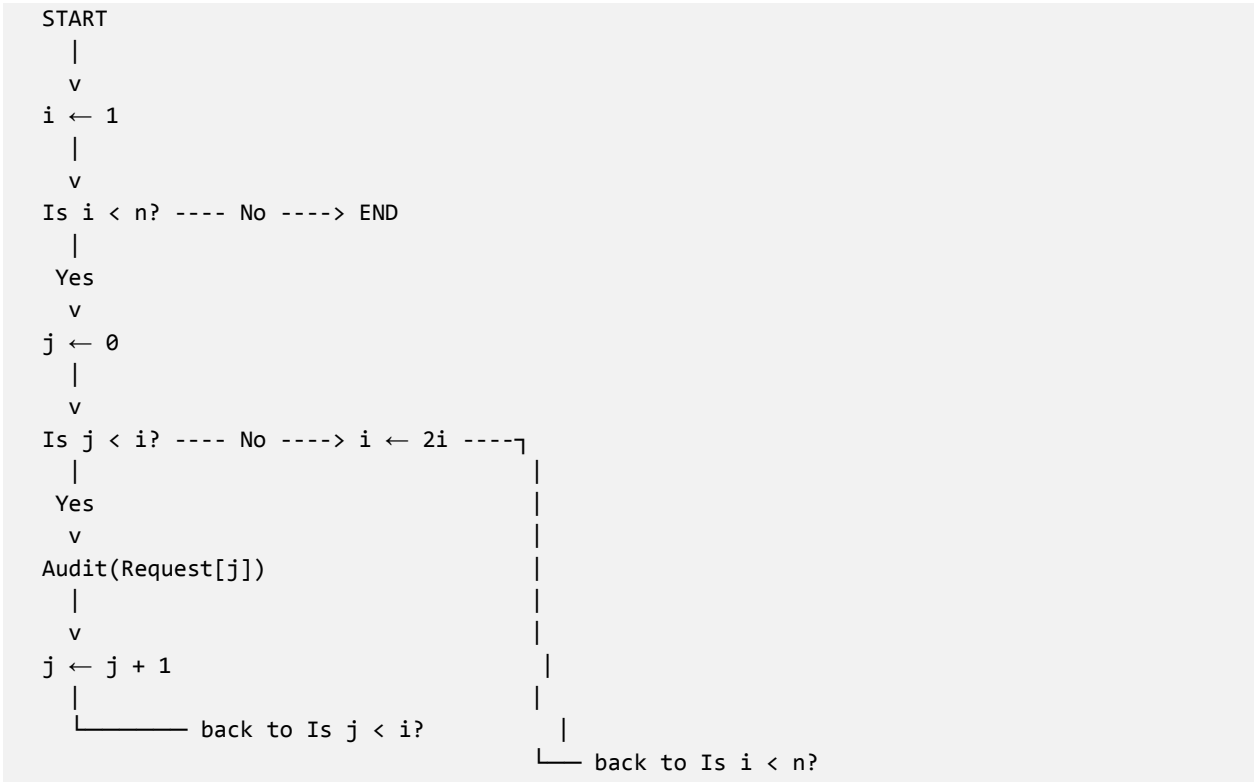
The values of i are 1, 2, 4, 8, …, 2^k, where $2^k < n$ and $k = \lfloor \log_2(n-1) \rfloor$. The inner loop performs i Audit operations on each pass.

```
A(n) = 1 + 2 + 4 + … + 2^k
     = 2^(k + 1) − 1
     = 2^(⌊log₂(n − 1)⌋ + 1) − 1, for n > 1
A(1) = 0
```

Since the largest power of two strictly below n is less than n and the geometric sum is less than 2n, $A(n) = \Theta(n)$. The team member's $\Theta(n \log n)$ claim is incorrect: the inner loop does not run n times on every outer iteration; its lengths form a geometric sequence.

Verification for n = 1,024: i takes values 1 through 512 (powers of two). Thus $A(1{,}024) = 1 + 2 + 4 + \ldots + 512 = 2^{10} - 1 = 1{,}023$ Audit operations.

M2 flowchart (control flow)



(c) Solve S2 by Backward Substitution

Given $T(n) = T(n/2) + 1$ and $T(1) = 1$, expand the recurrence repeatedly:

```
T(n) = T(n/2) + 1
     = [T(n/4) + 1] + 1
     = T(n/4) + 2
     = T(n/8) + 3
     = …
     = T(n/2^k) + k
```

Stop when $n/2^k = 1$. Therefore $n = 2^k$ and $k = \log_2 n$. Substituting the base case gives:

```
T(n) = T(1) + log₂n = 1 + log₂n
```

For n that is not an exact power of two, the comparison count is bounded by logarithmic growth; a standard halving search uses Θ(log n) comparisons. Hence S2 has Θ(log n) time complexity.

- Iterative binary search: Θ(log n) time and Θ(1) auxiliary space.
- Recursive binary search: Θ(log n) time and Θ(log n) call-stack space, because the recursion depth is logarithmic.

(d) Apply the Master Theorem

M4 – Current design: T(n) = 4T(n/2) + n

Identify a = 4, b = 2, and f(n) = n. Then n^(log_b a) = n^($\log_2 4$) = n^2. Since f(n) = n = O(n^(2 − 1)), f(n) is polynomially smaller than n^2. Master Theorem Case 1 applies, so T(n) = $\Theta(n^2)$.

M4 – Proposed redesign: T(n) = 2T(n/2) + n

Here a = 2, b = 2, and f(n) = n. Then n^(log_b a) = n^($\log_2 2$) = n. Since f(n) = Θ(n^(log_b a)), Case 2 applies, so T(n) = Θ(n log n).

When n doubles, the current quadratic design takes approximately four times as much work. For the redesigned n log n algorithm, the work grows by approximately $2\log_2(2n)/\log_2 n = 2(1 + 1/\log_2 n)$, which is slightly more than twice and approaches a factor of two as n grows.

M5 – T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, n^(log_b a) = n, and f(n) = n log n = $\Theta(n \log^1 n)$. The three basic Master Theorem cases in their simplest form do not directly cover this logarithmic factor. The extended Case 2 (or a recursion tree) gives T(n) = $\Theta(n \log^2 n)$.

Recursion-tree explanation: at each level, the total non-recursive work is Θ(n log(n/2^j)). There are Θ(log n) levels. Summing gives Θ(n) × Σ(log n − j) = $\Theta(n \log^2 n)$.

(e) Best, Average and Worst Cases for Search

S1 – Sequential search

- Best case: the product is the first item; 1 comparison.
- Average successful case: assuming the target is equally likely to be at any of the n positions, (1 + 2 + … + n)/n = (n + 1)/2 comparisons.
- Worst case: the product is last or absent; n comparisons.

If the searched product exists with probability p, and when it exists each position is equally likely, then:

```
E[C] = p × (n + 1)/2 + (1 − p) × n
```

For n = 1,000,000 and p = 0.5:

```
E[C] = 0.5 × 1,000,001/2 + 0.5 × 1,000,000
       = 250,000.25 + 500,000
       = 750,000.25 comparisons (expected value).
```

S2 – Binary search

- Best case: the target equals the first middle entry checked; 1 comparison.
- Average case: a successful target is typically found in Θ(log n) comparisons. The exact average depends on the tree shape and on the probability distribution of target keys; no distribution is specified in the question.

- Worst case: the search follows the deepest path or fails after narrowing the interval; $\Theta(\log n)$ comparisons. For n = 1,000,000, the usual halving implementation needs at most about 20 comparisons.

(f) Prove $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$

By definition, f(n) is $\Theta(n^2)$ if there are positive constants c_1, c_2 and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for every $n \ge n_0$.

Lower bound: $\Omega(n^2)$

For $n \ge 2$, the terms $10n \log_2 n$ and 50 are non-negative. Thus $f(n) = 3n^2 + 10n \log_2 n + 50 \ge 3n^2$. Choose $c_1 = 3$.

Upper bound: $O(n^2)$

For $n \ge 2$, $\log_2 n \le n$ and $50 \le 50n^2$. Therefore:

```
f(n) = 3n² + 10n log₂n + 50
     ≤ 3n² + 10n² + 50n²
     = 63n²
```

Choose $c_2 = 63$ and $n_0 = 2$. Hence $3n^2 \le f(n) \le 63n^2$ for all $n \ge 2$, proving $f(n) = \Theta(n^2)$.

Show that f(n) is not O(n)

Assume, for contradiction, that $f(n) = O(n)$. Then there must be constants $c > 0$ and n_1 such that $f(n) \le cn$ for all $n \ge n_1$. But $f(n) \ge 3n^2$, so this would require $3n^2 \le cn$, or $3n \le c$, for every sufficiently large n. This is impossible because 3n grows without bound. Therefore f(n) is not O(n).

(g) Scalability Decision for n = 1,000,000

The estimates below use the dominant asymptotic term as an operation-count proxy. Exact constants for M4 and M5 cannot be determined from the supplied recurrences because their base-case costs are not stated.

Module / design	Estimated operations	Estimated time at 10^8 ops/s	10-minute window?
M1 – duplicate detector	$n(n-1)/2 \approx 499{,}999{,}500{,}000$	≈ 4,999.995 s ≈ 83.3 min	FAIL
M2 – rate limiter	1,048,575	≈ 0.0105 s	PASS
S2 – binary search	≈ 20 comparisons	$\approx 2 \times 10^{-7}$ s	PASS
M4 – current	$\Theta(n^2)$, proxy $\approx 10^{12}$	proxy ≈ 10,000 s ≈ 166.7 min	FAIL
M4 – redesigned	$\Theta(n \log_2 n)$, proxy ≈ 19,931,569	proxy ≈ 0.199 s	PASS
M5 – analytics	$\Theta(n \log_2^2 n)$, proxy ≈ 397,267,000	proxy ≈ 3.97 s	PASS

M2 exact calculation at n = 1,000,000: the largest power of two less than n is 524,288, so the sum is $2 \times 524{,}288 - 1 = 1{,}048{,}575$ operations.

S2 estimate: $\lceil \log_2(1{,}000{,}000) \rceil \approx 20$ comparisons. Its precise count depends on the key and the implementation.

M4 redesigned estimate: $n \log_2 n \approx 1{,}000{,}000 \times 19.9316 \approx 19.93$ million units of work. M5 estimate: $n(\log_2 n)^2 \approx 1{,}000{,}000 \times 397.27 \approx 397.27$ million units of work. These are scaling proxies, not exact instruction counts.

Recommendations

- Redesign M1 first. Replace all-pairs comparisons with a hash-based grouping keyed by normalized (seller, product title), which can detect duplicates in expected $\Theta(n)$ time with additional $\Theta(n)$ storage. Define normalization carefully to avoid false matches.
- Redesign M4 before the catalogue reaches one million products. Remove replicated subproblems where possible; the proposed recurrence improves $\Theta(n^2)$ to $\Theta(n \log n)$.
- Keep M2's doubling-loop approach; its total work is $\Theta(n)$, and the claimed $\Theta(n \log n)$ bound should be corrected in the documentation.

- Keep S2's sorted-index approach and maintain the index efficiently as products change. It scales logarithmically per search.
- M5 is estimated to fit the 10-minute window under the stated operation-rate assumption, but profile it with real data because constants, I/O, and database overhead are omitted.
- The report generator f(n) is $\Theta(n^2)$, so it should also be monitored as the catalogue grows; its operation count is approximately 3×10^{12} at n = 1,000,000, which corresponds to about 30,000 seconds (500 minutes) if each counted operation maps directly to one server operation. It would therefore fail the 10-minute window and should be optimized.

Overall conclusion

At one million products, the clearly unacceptable quadratic workloads are M1 and the current M4 design. The report generator is also quadratic and, using its stated formula, is estimated at about 3.000×10^{12} operations (approximately 30,000 seconds at 10^8 operations/second), so it must be redesigned or distributed. Prioritize expected-linear duplicate detection, the M4 replication removal, and report-generator optimization. M2 and S2 scale well; M5's $\Theta(n \log^2 n)$ estimate is within the window under the simplified model.

Submission note

Review the assumptions and adapt the explanations to your own understanding before submission. The assignment's question sheet does not specify recurrence base-case costs or exact machine-level constants, so recurrence-based running-time figures are asymptotic estimates rather than exact execution times.